

Convenciones del esquema

Antes de ver las tablas, las reglas que aplican a todo el esquema sin excepción.

Todos los identificadores son UUID v4, no enteros autoincrement. La razón es multi-tenant: los IDs nunca se filtran entre establecimientos, pero si usaras enteros secuenciales un usuario podría inferir cuántos registros tiene otro establecimiento por el número de su propio ID. Con UUID eso es imposible. También simplifica la generación de IDs en el cliente para operaciones optimistas.

Todos los timestamps son timestamptz (timestamp with time zone) almacenados en UTC. Paraguay está en UTC-4 (sin cambio de horario). La conversión a hora local es responsabilidad del frontend, nunca de la base de datos.

Todos los campos de texto que son enumeraciones usan tipos TEXT con constraints CHECK en lugar de tipos ENUM de PostgreSQL. La razón es que agregar un valor a un ENUM de PostgreSQL requiere un ALTER TYPE que en algunos escenarios bloquea la tabla. Con CHECK constraint es un simple ALTER TABLE.

Los campos monetarios se almacenan como NUMERIC(15, 2), nunca como FLOAT. Los floats tienen errores de precisión en aritmética decimal. En un sistema financiero eso es inaceptable.

Bloque 1 — Fundación multi-tenant

sql

-- _____

-- EXTENSIONES

-- _____

create extension if not exists "uuid-ossf";

create extension if not exists "pgcrypto";

-- _____

-- ESTABLECIMIENTOS

-- Raíz del sistema. Todo lo demás referencia esta tabla.

-- _____

```
create table establecimientos (  
  id          uuid primary key default uuid_generate_v4(),  
  nombre      text not null,  
  departamento text,  
  coordenadas_lat  numeric(10, 7),  
  coordenadas_lng  numeric(10, 7),  
  numero_senacsa   text unique,  
  nombre_propietario text not null,  
  fecha_inicio_sistema date not null,  
  ejercicio_inicio_mes integer not null default 7 -- julio en Paraguay  
    check (ejercicio_inicio_mes between 1 and 12),  
  activo        boolean not null default true,  
  created_at     timestampz not null default now(),  
  updated_at     timestampz not null default now()  
);
```

-- _____

-- USUARIOS Y SU RELACIÓN CON ESTABLECIMIENTOS

-- Un usuario puede pertenecer a varios establecimientos (V2).

-- En MVP la UI solo expone uno, pero la tabla ya soporta N.

-- _____

```

create table usuarios_establecimientos (
  id          uuid primary key default uuid_generate_v4(),
  user_id     uuid not null references auth.users(id) on delete cascade,
  establecimiento_id uuid not null references establecimientos(id) on delete cascade,
  rol         text not null
    check (rol in ('propietario', 'administrador', 'veterinario')),
  activo      boolean not null default true,
  created_at   timestamptz not null default now(),
  unique (user_id, establecimiento_id)
);

```

```

create index idx_ue_user_id on usuarios_establecimientos(user_id);

```

```

create index idx_ue_establecimiento_id on
usuarios_establecimientos(establecimiento_id);

```

```

-- _____

```

```

-- TIPO DE CAMBIO DE REFERENCIA

```

```

-- El operador puede configurar una tasa PYG/USD.

```

```

-- Se almacena el historial — nunca se sobrescribe.

```

```

-- _____

```

```

create table tipos_de_cambio (
  id          uuid primary key default uuid_generate_v4(),
  establecimiento_id uuid not null references establecimientos(id),
  tasa_pyg_por_usd numeric(12, 2) not null,
  fecha_vigencia date not null,

```

```
usuario_id    uuid references auth.users(id),
created_at    timestampz not null default now(),
unique (establecimiento_id, fecha_vigencia)
);
```

Bloque 2 — Configuración del establecimiento

sql

```
-- -----
-- CATEGORÍAS Y COEFICIENTES UG
-- Precargadas con valores estándar paraguayos.
-- El operador puede ajustar los coeficientes.
-- -----

create table categorias (
  id          uuid primary key default uuid_generate_v4(),
  establecimiento_id uuid not null references establecimientos(id),
  nombre      text not null
  check (nombre in (
    'ternero', 'ternera', 'novillo', 'vaquillona',
    'toro', 'vaca', 'vaca_con_cria', 'buey'
  )),
  coeficiente_ug numeric(4, 2) not null,
  activo        boolean not null default true,
  created_at    timestampz not null default now(),
  updated_at    timestampz not null default now(),
  unique (establecimiento_id, nombre)
);
```

```
create index idx_categorias_establecimiento on categorias(establecimiento_id);
```

```
-- _____
```

```
-- POTREROS
```

```
-- Unidad de gestión forrajera.
```

```
-- Nunca se borran — se archivan.
```

```
-- _____
```

```
create table potreros (
```

```
  id          uuid primary key default uuid_generate_v4(),
```

```
  establecimiento_id  uuid not null references establecimientos(id),
```

```
  nombre           text not null,
```

```
  superficie_ha     numeric(10, 2),
```

```
  tipo_pastura      text,
```

```
  capacidad_max_ug_ha  numeric(6, 2),
```

```
  estado           text not null default 'activo'
```

```
    check (estado in ('activo', 'en_descanso', 'en_recuperacion', 'archivado')),
```

```
  created_at       timestamptz not null default now(),
```

```
  updated_at       timestamptz not null default now(),
```

```
  unique (establecimiento_id, nombre)
```

```
);
```

```
create index idx_potreros_establecimiento on potreros(establecimiento_id);
```

```
create index idx_potreros_estado on potreros(establecimiento_id, estado);
```

-- _____

-- LOTES

-- Unidad de gestión operativa y económica.

-- El cierre es automático — ver trigger más adelante.

-- _____

```
create table lotes (  
  id          uuid primary key default uuid_generate_v4(),  
  establecimiento_id  uuid not null references establecimientos(id),  
  nombre      text not null,  
  proposito   text not null  
    check (proposito in ('invernada', 'cria', 'recria', 'reproduccion')),  
  potrero_principal_id  uuid references potreros(id),  
  fecha_formacion  date not null,  
  fecha_cierre     date,  
  peso_promedio_ingreso numeric(7, 2),  
  peso_objetivo_salida numeric(7, 2),  
  plazo_estimado_dias integer,  
  estado          text not null default 'activo'  
    check (estado in ('activo', 'cerrado')),  
  created_at      timestamptz not null default now(),  
  updated_at      timestamptz not null default now()  
);  
  
create index idx_lotes_establecimiento on lotes(establecimiento_id);  
create index idx_lotes_estado on lotes(establecimiento_id, estado);
```

Bloque 3 — Animales e identidad

sql

-- _____

-- ANIMALES

-- Entidad central. Nunca se borran (RN-01).

-- Los atributos derivados NO se almacenan aquí.

-- _____

create table animales (

id uuid primary key default uuid_generate_v4(),

establecimiento_id uuid not null references establecimientos(id),

-- identidad

caravana_senacsa text,

numero_campo text,

-- al menos uno de los dos debe existir (validado en aplicación)

-- zootécnico

raza text,

sexo text not null

check (sexo in ('macho', 'hembra')),

fecha_nacimiento date,

fecha_nacimiento_estimada boolean not null default false,

-- origen

tipo_origen text not null

```

    check (tipo_origen in ('nacido', 'comprado')),
establecimiento_origen text,
madre_id      uuid references animales(id),
padre_id      uuid references animales(id),

-- estado
estado        text not null default 'activo'

    check (estado in ('activo', 'egresado')),
fecha_egreso  date,
tipo_egreso   text

    check (tipo_egreso in ('venta', 'faena', 'muerte')),

-- lote y potrero actuales (desnormalización controlada para performance)
-- la fuente de verdad es la tabla de eventos de movimiento
-- estos campos se actualizan con cada movimiento via trigger
lote_actual_id  uuid references lotes(id),
potrero_actual_id  uuid references potreros(id),

created_at      timestampz not null default now(),
updated_at      timestampz not null default now(),

-- un animal activo necesita identificación
constraint animal_tiene_identificacion

    check (caravana_senacsa is not null or numero_campo is not null),

unique (establecimiento_id, caravana_senacsa),

```



```
unique (establecimiento_id, numero_campo)
);
```

```
create index idx_animales_establecimiento on animales(establecimiento_id);
create index idx_animales_estado on animales(establecimiento_id, estado);
create index idx_animales_lote on animales(lote_actual_id);
create index idx_animales_potrero on animales(potrero_actual_id);
create index idx_animales_caravana on animales(establecimiento_id,
caravana_senacsa)
where caravana_senacsa is not null;
create index idx_animales_madre on animales(madre_id)
where madre_id is not null;
```

```
-- _____
```

```
-- HISTORIAL DE CATEGORÍAS
```

```
-- La categoría actual es la que tiene fecha_fin null.
```

```
-- Implementa RN-05 e IND-09.
```

```
-- _____
```

```
create table animal_categorias (
id      uuid primary key default uuid_generate_v4(),
animal_id  uuid not null references animales(id),
categoria  text not null
check (categoria in (
'ternero', 'ternera', 'novillo', 'vaquillona',
'toro', 'vaca', 'vaca_con_cria', 'buey'
```

```

)),
fecha_inicio date not null,
fecha_fin date,
usuario_id uuid references auth.users(id),
created_at timestamptz not null default now(),

-- solo una categoría activa por animal
constraint una_categoria_activa
exclude using gist (
    animal_id with =,
    daterange(fecha_inicio, fecha_fin, '[') with &&
)
);

create index idx_categorias_animal on animal_categorias(animal_id);
create index idx_categorias_activa on animal_categorias(animal_id)
where fecha_fin is null;

```

Bloque 4 — Eventos (estructura base e inmutabilidad)

sql

```

-- -----

-- EVENTOS — tabla base

-- Todos los eventos del sistema heredan de esta estructura.
-- Implementa RN-15 (inmutabilidad) y RN-16 (doble fecha).
-- -----

create table eventos (

```

```

id          uuid primary key default uuid_generate_v4(),
establecimiento_id uuid not null references establecimientos(id),
tipo        text not null

check (tipo in (
    'movimiento', 'pesaje', 'sanitario_vacunacion',
    'sanitario_tratamiento', 'sanitario_diagnostico',
    'reproductivo_servicio', 'reproductivo_diagnostico_prenez',
    'reproductivo_paricion', 'cambio_categoria'
)),

fecha_evento    date not null,
fecha_registro  timestampz not null default now(),
usuario_id      uuid not null references auth.users(id),
observaciones   text,

-- inmutabilidad: si este evento fue corregido por otro
corregido_por_id  uuid references eventos(id),
es_correccion_de_id uuid references eventos(id),
anulado          boolean not null default false,

created_at      timestampz not null default now(),

-- RN-18: no se permiten fechas futuras
constraint fecha_evento_no_futura
    check (fecha_evento <= current_date)
);

```

-- inmutabilidad a nivel de base de datos

create rule eventos_no_update as

on update to eventos do instead nothing;

create index idx_eventos_establecimiento on eventos(establecimiento_id);

create index idx_eventos_tipo on eventos(establecimiento_id, tipo);

create index idx_eventos_fecha on eventos(establecimiento_id, fecha_evento desc);

-- _____

-- EVENTOS_ANIMALES — relación entre evento y animal

-- Un evento puede afectar a uno o muchos animales.

-- Cuando se registra una vacunación de lote, se crea

-- un registro aquí por cada animal (RN-17).

-- _____

create table eventos_animales (

id uuid primary key default uuid_generate_v4(),

evento_id uuid not null references eventos(id),

animal_id uuid not null references animales(id),

-- referencia al evento de lote que originó este evento individual

evento_lote_id uuid references eventos(id),

created_at timestamptz not null default now(),

unique (evento_id, animal_id)

);

create index idx_ea_evento on eventos_animales(evento_id);

```
create index idx_ea_animal on eventos_animales(animal_id);

create index idx_ea_animal_tipo
  on eventos_animales(animal_id)
  include (evento_id);
```

Bloque 5 — Especializaciones de eventos

sql

```
-- -----

-- PESAJES

-- -----

create table evento_pesajes (
  evento_id  uuid primary key references eventos(id),
  animal_id  uuid references animales(id), -- null si es pesaje de lote
  lote_id    uuid references lotes(id),   -- null si es individual
  tipo       text not null
    check (tipo in ('individual', 'lote_estimado')),
  peso_kg    numeric(7, 2) not null
    check (peso_kg > 0),
  cantidad_muestra integer, -- solo para pesaje de lote
  -- GDP calculado y almacenado en el momento del pesaje
  -- (la fuente de verdad del pesaje anterior queda fija)
  gdp_g_dia  numeric(8, 2),
  dias_intervalo integer
);

create index idx_pesajes_animal on evento_pesajes(animal_id)
```

```
where animal_id is not null;

create index idx_pesajes_lote on evento_pesajes(lote_id)

where lote_id is not null;
```

```
-- _____
```

```
-- MOVIMIENTOS
```

```
-- _____
```

```
create table evento_movimientos (

evento_id      uuid primary key references eventos(id),

tipo_movimiento  text not null

check (tipo_movimiento in (

    'ingreso_compra', 'nacimiento', 'traslado_interno',

    'egreso_venta', 'egreso_faena', 'egreso_muerte'

)),
```

```
-- campos comunes de ubicación
```

```
potrero_origen_id  uuid references potreros(id),

potrero_destino_id  uuid references potreros(id),

lote_destino_id     uuid references lotes(id),
```

```
-- ingreso por compra
```

```
establecimiento_origen text,

numero_guia_senacsa  text,

proveedor            text,

precio_unitario      numeric(15, 2),
```

tipo_precio text

check (tipo_precio in ('por_cabeza', 'por_kg')),

moneda text

check (moneda in ('PYG', 'USD')),

-- egreso por venta o faena

comprador text,

destino_venta text

check (destino_venta in ('frigorifico', 'remate', 'venta_directa')),

precio_venta_unitario numeric(15, 2),

peso_venta_promedio_kg numeric(7, 2),

-- egreso por muerte

causa_muerte text,

-- nacimiento

padre_id uuid references animales(id)

);

create index idx_movimientos_tipo on evento_movimientos(tipo_movimiento);

create index idx_movimientos_guia on evento_movimientos(numero_guia_senacsa)

where numero_guia_senacsa is not null;

-- _____

-- VACUNACIONES

```

create table evento_vacunaciones (
    evento_id      uuid primary key references eventos(id),
    biologico      text not null,
    laboratorio     text,
    numero_lote_biologico text,
    fecha_vencimiento_biol date,
    dosis_ml       numeric(6, 2),
    via_administracion text
    check (via_administracion in (
        'subcutanea', 'intramuscular', 'intravenosa', 'oral', 'topica'
    )),
    -- indica si fue la vacuna antiaftosa (para el indicador IND-14)
    es_antiaftosa   boolean not null default false,
    lote_id         uuid references lotes(id) -- si fue vacunación de lote
);

```

```

-- TRATAMIENTOS VETERINARIOS
-- Implementa RN-06, RN-07 y RN-08.

```

```

create table evento_tratamientos (
    evento_id      uuid primary key references eventos(id),
    animal_id      uuid not null references animales(id),
    diagnostico     text,

```



```

medicamento      text not null,
dosis             text,
via_administracion text

check (via_administracion in (
    'subcutanea', 'intramuscular', 'intravenosa', 'oral', 'topica'
)),
duracion_dias     integer,
dias_carencia     integer not null default 0,
fecha_fin_carencia date not null,
veterinario       text,
-- estado calculado: activo si fecha_fin_carencia >= hoy y evento no anulado
-- no se almacena — se calcula en query
costo             numeric(15, 2),
moneda_costo      text

check (moneda_costo in ('PYG', 'USD'))
);

```

```

create index idx_tratamientos_animal on evento_tratamientos(animal_id);

create index idx_tratamientos_carencia on evento_tratamientos(animal_id,
fecha_fin_carencia)

where fecha_fin_carencia >= current_date;

```

```

-- _____

-- DIAGNÓSTICOS VETERINARIOS

-- _____

```

```
create table evento_diagnostics (  
  evento_id    uuid primary key references eventos(id),  
  animal_id    uuid not null references animales(id),  
  descripcion  text not null,  
  veterinario  text,  
  con_tratamiento boolean not null default false,  
  tratamiento_id uuid references evento_tratamientos(evento_id)  
);
```

```
-- _____
```

```
-- EVENTOS REPRODUCTIVOS — SERVICIO
```

```
-- _____
```

```
create table evento_servicios (  
  evento_id      uuid primary key references eventos(id),  
  hembra_id      uuid not null references animales(id),  
  tipo_servicio   text not null  
  check (tipo_servicio in ('monta_natural', 'inseminacion_artificial')),  
  toro_id        uuid references animales(id),  
  codigo_semen    text,  
  fecha_prevista_diagnostico date  
);
```

```
-- _____
```

```
-- EVENTOS REPRODUCTIVOS — DIAGNÓSTICO DE PREÑEZ
```

```
create table evento_diagnosticos_prenez (  
  evento_id      uuid primary key references eventos(id),  
  hembra_id      uuid not null references animales(id),  
  servicio_id     uuid references evento_servicios(evento_id),  
  resultado       text not null  
  check (resultado in ('positivo', 'negativo')),  
  metodo          text  
  check (metodo in ('tacto_rectal', 'ecografia')),  
  veterinario     text,  
  fecha_probable_parto date -- calculado: fecha_servicio + 283 días  
);
```

-- EVENTOS REPRODUCTIVOS — PARICIÓN

```
create table evento_pariciones (  
  evento_id      uuid primary key references eventos(id),  
  madre_id       uuid not null references animales(id),  
  cria_id        uuid references animales(id), -- creado automáticamente  
  tipo_parto     text  
  check (tipo_parto in ('normal', 'distocico', 'asistido')),  
  estado_cria    text  
  check (estado_cria in ('vivo', 'nacido_muerto'))  
);
```

Bloque 6 — Eventos económicos

sql

```
-- -----  
  
-- EVENTOS ECONÓMICOS  
  
-- Componente financiero que acompaña a ciertos movimientos.  
-- Implementa IND-15, IND-16 e IND-17.  
  
-- -----  
  
create table eventos_economicos (  
  id          uuid primary key default uuid_generate_v4(),  
  establecimiento_id uuid not null references establecimientos(id),  
  evento_id   uuid references eventos(id),  
  animal_id   uuid references animales(id),  
  lote_id     uuid references lotes(id),  
  tipo        text not null  
    check (tipo in (  
      'compra_animal', 'venta_animal', 'insumo_sanitario',  
      'honorario_veterinario', 'otro_costo'  
    )),  
  monto       numeric(15, 2) not null,  
  moneda      text not null  
    check (moneda in ('PYG', 'USD')),  
  tasa_cambio_ref numeric(12, 2), -- tasa vigente al momento del registro  
  descripcion  text,  
  created_at   timestampz not null default now()  
);
```

```
create index idx_ee_animal on eventos_economicos(animal_id)
  where animal_id is not null;

create index idx_ee_lote on eventos_economicos(lote_id)
  where lote_id is not null;

create index idx_ee_establecimiento on eventos_economicos(establecimiento_id);
```

Bloque 7 — Alertas

sql

```
-- -----

-- ALERTAS

-- En el MVP se calculan en el momento (no se persisten).
-- Esta tabla existe para V2 cuando el cálculo sea asíncrono.
-- Se deja el esquema para que la migración sea un ALTER, no un CREATE.
-- -----

create table alertas (
  id          uuid primary key default uuid_generate_v4(),
  establecimiento_id uuid not null references establecimientos(id),
  tipo        text not null
  check (tipo in (
    'antiaftosa_vencida', 'carencia_activa', 'vacunacion_proxima',
    'potrero_sobrecargado', 'sin_pesaje_reciente', 'gdp_bajo',
    'dias_sistema_excedidos', 'peso_objetivo_alcanzado'
  )),
  severidad   text not null
  check (severidad in ('critica', 'preventiva', 'informativa')),
```

```

    animal_id      uuid references animales(id),
    lote_id        uuid references lotes(id),
    potrero_id     uuid references potreros(id),
    mensaje        text not null,
    activa         boolean not null default true,
    resuelta_at    timestampz,
    created_at     timestampz not null default now(),
    updated_at     timestampz not null default now()
);

create index idx_alertas_establecimiento on alertas(establecimiento_id, activa)
where activa = true;

```

Bloque 8 — Importaciones

sql

```

-- -----
-- IMPORTACIONES CSV
-- Registro de cada importación masiva.
-- Permite reprocesar y auditar el onboarding.
-- -----

create table importaciones (
    id              uuid primary key default uuid_generate_v4(),
    establecimiento_id uuid not null references establecimientos(id),
    usuario_id      uuid not null references auth.users(id),
    nombre_archivo  text not null,
    total_filas     integer,

```

```
filas_exitosas integer,  
filas_con_error integer,  
estado text not null default 'procesando'  
    check (estado in ('procesando', 'completado', 'completado_con_errores', 'fallido')),  
reporte_errores jsonb, -- array de {fila, campo, mensaje}  
created_at timestampz not null default now(),  
completado_at timestampz  
);
```

Bloque 9 — Row Level Security

sql

```
-- -----  
  
-- ROW LEVEL SECURITY  
  
-- Aplica a todas las tablas de dominio.  
-- El patrón se repite: el usuario solo ve datos  
-- de establecimientos a los que pertenece y está activo.  
-- -----  
  
-- función auxiliar reutilizable  
  
create or replace function mis_establecimientos()  
returns setof uuid  
  
language sql security definer stable as $$  
  
    select establecimiento_id  
  
    from usuarios_establecimientos  
  
    where user_id = auth.uid()  
  
    and activo = true
```

\$\$;

-- habilitar RLS en todas las tablas de dominio

alter table establecimientos enable row level security;

alter table potreros enable row level security;

alter table lotes enable row level security;

alter table animales enable row level security;

alter table animal_categorias enable row level security;

alter table eventos enable row level security;

alter table eventos_animales enable row level security;

alter table evento_pesajes enable row level security;

alter table evento_movimientos enable row level security;

alter table evento_vacunaciones enable row level security;

alter table evento_tratamientos enable row level security;

alter table evento_diagnosticos enable row level security;

alter table evento_servicios enable row level security;

alter table evento_diagnosticos_prenez enable row level security;

alter table evento_pariciones enable row level security;

alter table eventos_economicos enable row level security;

alter table alertas enable row level security;

alter table importaciones enable row level security;

-- políticas — patrón estándar por tabla con establecimiento_id

create policy "acceso por establecimiento" on establecimientos

for all using (id in (select mis_establecimientos()));


```
create policy "acceso por establecimiento" on potreros
for all using (establecimiento_id in (select mis_establecimientos()));
```

```
create policy "acceso por establecimiento" on lotes
for all using (establecimiento_id in (select mis_establecimientos()));
```

```
create policy "acceso por establecimiento" on animales
for all using (establecimiento_id in (select mis_establecimientos()));
```

```
create policy "acceso por establecimiento" on eventos
for all using (establecimiento_id in (select mis_establecimientos()));
```

```
create policy "acceso por establecimiento" on eventos_economicos
for all using (establecimiento_id in (select mis_establecimientos()));
```

```
create policy "acceso por establecimiento" on alertas
for all using (establecimiento_id in (select mis_establecimientos()));
```

```
create policy "acceso por establecimiento" on importaciones
for all using (establecimiento_id in (select mis_establecimientos()));
```

```
-- para tablas que no tienen establecimiento_id directo,
```

```
-- el acceso se valida por join con la tabla padre
```

```
create policy "acceso via animal" on animal_categorias
for all using (
    animal_id in (
```

```
select id from animales
where establecimiento_id in (select mis_establecimientos())
)
);
```

```
create policy "acceso via evento" on eventos_animales
for all using (
evento_id in (
select id from eventos
where establecimiento_id in (select mis_establecimientos())
)
);
```

Bloque 10 — Funciones y triggers críticos

sql

```
-- -----
-- TRIGGER: actualizar lote_actual_id y potrero_actual_id en animales
-- Se dispara después de insertar un evento de movimiento.
-- Mantiene la desnormalización consistente.
```

```
-- -----
create or replace function actualizar_ubicacion_animal()
returns trigger language plpgsql as $$
begin
-- traslado interno
if NEW.tipo_movimiento = 'traslado_interno' then
update animales
```

```
set potrero_actual_id = NEW.potrero_destino_id,
```

```
    updated_at = now()
```

```
where id in (
```

```
    select animal_id from eventos_animales
```

```
    where evento_id = NEW.evento_id
```

```
);
```

```
-- ingreso
```

```
elsif NEW.tipo_movimiento in ('ingreso_compra', 'nacimiento') then
```

```
    update animales
```

```
set potrero_actual_id = NEW.potrero_destino_id,
```

```
    lote_actual_id = NEW.lote_destino_id,
```

```
    updated_at = now()
```

```
where id in (
```

```
    select animal_id from eventos_animales
```

```
    where evento_id = NEW.evento_id
```

```
);
```

```
-- egreso
```

```
elsif NEW.tipo_movimiento in ('egreso_venta', 'egreso_faena', 'egreso_muerte') then
```

```
    update animales
```

```
set estado = 'egresado',
```

```
    fecha_egreso = (
```

```
        select fecha_evento from eventos where id = NEW.evento_id
```

```
    ),
```

```
    tipo_egreso = case NEW.tipo_movimiento
```

```

        when 'egreso_venta' then 'venta'

        when 'egreso_faena' then 'faena'

        when 'egreso_muerte' then 'muerte'

    end,

    potrero_actual_id = null,

    lote_actual_id  = null,

    updated_at      = now()

where id in (

    select animal_id from eventos_animales

    where evento_id = NEW.evento_id

);

end if;


return NEW;

end;

$$;


create trigger trg_actualizar_ubicacion

after insert on evento_movimientos

for each row execute function actualizar_ubicacion_animal();


-- -----

-- TRIGGER: cierre automático de lote (RN-09)

-- Después de cada egreso verifica si el lote quedó vacío.

-- -----

```

```
create or replace function verificar_cierre_lote()
returns trigger language plpgsql as $$
declare
    v_lote_id uuid;
    v_fecha_evento date;
    v_animales_activos integer;
begin
    -- obtener el lote de cada animal afectado por el egreso
    for v_lote_id in
        select distinct a.lote_actual_id
        from eventos_animales ea
        join animales a on a.id = ea.animal_id
        where ea.evento_id = NEW.evento_id
        and a.lote_actual_id is not null
    loop
        select count(*) into v_animales_activos
        from animales
        where lote_actual_id = v_lote_id
        and estado = 'activo';

        if v_animales_activos = 0 then
            select fecha_evento into v_fecha_evento
            from eventos where id = NEW.evento_id;

            update lotes
            set estado = 'cerrado',
```

```
        fecha_cierre = v_fecha_evento,  
        updated_at = now()  
    where id = v_lote_id;  
end if;  
end loop;
```

```
return NEW;  
end;  
$$;
```

```
create trigger trg_cierre_lote  
after insert on evento_movimientos  
for each row  
when (NEW.tipo_movimiento in ('egreso_venta', 'egreso_faena', 'egreso_muerte'))  
execute function verificar_cierre_lote();
```

```
-- _____
```

```
-- FUNCIÓN: verificar carencia activa (RN-06 y RN-08)
```

```
-- Llamada desde el backend antes de confirmar un egreso por venta.
```

```
-- Devuelve los animales bloqueados con la fecha de fin de carencia.
```

```
-- _____
```

```
create or replace function animales_con_carencia_activa(  
    p_animal_ids uuid[]  
)
```

```
returns table (
```

```

    animal_id    uuid,
    fecha_fin_carencia date,
    medicamento   text
)
language sql stable as $$
select
    et.animal_id,
    max(et.fecha_fin_carencia) as fecha_fin_carencia,
    string_agg(et.medicamento, ', ') as medicamento
from evento_tratamientos et
join eventos e on e.id = et.evento_id
where et.animal_id = any(p_animal_ids)
    and et.fecha_fin_carencia >= current_date
    and e.anulado = false
group by et.animal_id
having max(et.fecha_fin_carencia) >= current_date;
$$;

```

-- FUNCIÓN: calcular GDP (IND-01)

-- Llamada desde el backend al registrar un pesaje.

create or replace function calcular_gdp(

p_animal_id uuid

)

```

returns table (
    gdp_g_dia    numeric,
    peso_anterior numeric,
    dias_intervalo integer
)
language sql stable as $$
with ultimos_dos as (
    select
        ep.peso_kg,
        e.fecha_evento,
        row_number() over (order by e.fecha_evento desc) as rn
    from evento_pesajes ep
    join eventos e on e.id = ep.evento_id
    where ep.animal_id = p_animal_id
        and ep.tipo = 'individual'
        and e.anulado = false
    order by e.fecha_evento desc
    limit 2
)
select
    case
        when (max(case when rn=1 then fecha_evento end) -
            max(case when rn=2 then fecha_evento end)) = 0
        then null -- división por cero
        else round(
            (max(case when rn=1 then peso_kg end) -

```



```

        max(case when rn=2 then peso_kg end)) /
        (max(case when rn=1 then fecha_evento end) -
        max(case when rn=2 then fecha_evento end)) * 1000,
        2
    )
end as gdp_g_dia,
max(case when rn=2 then peso_kg end) as peso_anterior,
(max(case when rn=1 then fecha_evento end) -
max(case when rn=2 then fecha_evento end))::integer as dias_intervalo
from ultimos_dos
having count(*) = 2;
$$;

```

Datos iniciales

sql

```

-- -----
-- SEED: coeficientes UG estándar paraguayos
-- Se insertan para cada establecimiento al momento del onboarding.
-- Esta función se llama desde el backend al crear un establecimiento.
-- -----

create or replace function seed_categorias_establecimiento(
    p_establecimiento_id uuid
)
returns void language sql as $$

insert into categorias (establecimiento_id, nombre, coeficiente_ug) values
    (p_establecimiento_id, 'ternero', 0.30),

```

```
(p_establecimiento_id, 'ternera', 0.30),  
(p_establecimiento_id, 'novillo', 0.70),  
(p_establecimiento_id, 'vaquillona', 0.70),  
(p_establecimiento_id, 'vaca', 1.00),  
(p_establecimiento_id, 'vaca_con_cria', 1.20),  
(p_establecimiento_id, 'toro', 1.20),  
(p_establecimiento_id, 'buey', 1.00);  
$;
```

Decisiones que merecen explicación

¿Por qué lote_actual_id y potrero_actual_id están desnormalizados en animales?

La fuente de verdad es el historial de eventos de movimiento. Pero si cada vez que el dashboard carga necesitás recorrer todos los eventos de 2000 animales para saber dónde están, la performance va a ser inaceptable. La desnormalización controlada con triggers garantiza consistencia y velocidad. Es un patrón clásico y correcto para este caso.

¿Por qué el GDP se almacena en evento_pesajes si es un derivado?

Porque el GDP depende del pesaje anterior en el momento de registro. Si mañana anulás el pesaje anterior por un error, el GDP de todos los pesajes posteriores quedaría inconsistente si lo recalculáramos. Al almacenarlo en el momento del registro con los datos que existían entonces, mantenés un registro fiel de lo que se calculó y cuándo. Esto es coherente con la inmutabilidad de los eventos.

¿Por qué JSONB para el reporte de errores de importación?

Porque la estructura de errores puede variar: algunos tienen número de fila, otros tienen nombre de campo, otros tienen mensajes largos. JSONB es flexible y consultable. No necesitás una tabla separada de errores de importación con columnas fijas para algo que solo se lee ocasionalmente.

¿Por qué la función mis_establecimientos() con security definer?

Porque las políticas RLS se evalúan para cada fila de cada query. Si cada política hace un subquery a usuarios_establecimientos, eso puede ser costoso. La función con

security definer y stable le permite a PostgreSQL cachearla dentro de la transacción, ejecutándola una sola vez por query en lugar de una vez por fila.